

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 06 -  
**CI/CD PARA  
ARQUITETURAS  
SERVERLESS**

---

## SUMÁRIO

|                                   |    |
|-----------------------------------|----|
| O QUE VEM POR AÍ? .....           | 3  |
| HANDS ON .....                    | 4  |
| SAIBA MAIS.....                   | 5  |
| MERCADO, CASES E TENDÊNCIAS ..... | 10 |
| O QUE VOCÊ VIU NESTA AULA? .....  | 11 |
| REFERÊNCIAS.....                  | 12 |

EMANDA

## O QUE VEM POR AÍ?

E aí, pessoal! Nesta aula, vamos mergulhar no CI/CD para arquiteturas serverless. A ideia é ir além dos conceitos e mostrar, na prática, como construir pipelines que nos liberam do gerenciamento de servidores para que possamos focar no que realmente importa: o código.

Você vai aprender a usar o AWS Serverless Application Model (AWS SAM) para automatizar cada etapa, do build ao deploy, garantindo que suas aplicações cheguem em produção de forma rápida e segura. Prepare-se para colocar a mão na massa!

## HANDS ON

A teoria vira prática nas nossas videoaulas. Você será guiado(a) passo a passo para construir um pipeline de CI/CD pronto para o mercado. Na primeira aula, focaremos no build e nos testes com o comando `sam build`; na segunda, vamos gerenciar o deploy para ambientes de staging e production com o `sam deploy`.

O objetivo é que você se familiarize com um fluxo de trabalho profissional e automatizado, em que cada passo é rastreável e repetível.

## SAIBA MAIS

### A Essência do Serverless: Da Abstração à Agilidade

A arquitetura serverless representa um salto de paradigma na computação em nuvem. A popularização do termo "sem servidor" muitas vezes gera a percepção incorreta de que as aplicações serverless operam sem uma infraestrutura subjacente. A realidade, no entanto, é que o modelo serverless é uma camada de abstração que oculta a complexidade do gerenciamento de servidores, sistemas operacionais e patches de segurança. O provedor de nuvem, como a Amazon Web Services (AWS), é o responsável por todo o trabalho operacional de provisionamento, manutenção, dimensionamento e balanceamento de carga, o que nos libera para focarmos exclusivamente na criação de valor por meio da lógica de negócio.

Essa abordagem traz benefícios estratégicos consideráveis. O aumento da produtividade da pessoa desenvolvedora é um dos mais evidentes, pois a equipe pode gastar menos tempo com tarefas de infraestrutura e mais tempo com a inovação. A escalabilidade automática, um dos pilares da arquitetura serverless, garante que as aplicações possam se adaptar instantaneamente a picos de tráfego, sem a necessidade de intervenção manual. E o modelo de cobrança de pagamento por uso, em que o custo é atrelado apenas ao tempo de execução e à quantidade de recursos consumidos, elimina o desperdício de recursos ociosos e alinha os custos ao uso real da aplicação.

O termo serverless abrange uma variedade de modelos de computação. O mais proeminente é o Function-as-a-Service (FaaS), exemplificado pelo AWS Lambda, em que a aplicação é decomposta em pequenas funções. Já o Backend-as-a-Service (BaaS) se refere à utilização de serviços gerenciados para funcionalidades prontas para uso, como bancos de dados ou autenticação. Por fim, os serverless containers (por exemplo, AWS Fargate, Google Cloud Run) oferecem um modelo de abstração que permite a execução de contêineres sem a necessidade de gerenciar a infraestrutura de orquestração.

## A Abordagem de CI/CD para Arquiteturas Serverless

Se você já trabalhou com CI/CD em ambientes tradicionais, sabe que o pipeline geralmente cuida da compilação do código, da criação da imagem e da implantação em servidores. Com a arquitetura serverless, esse processo muda. Como nós não gerenciamos a infraestrutura, o pipeline se torna uma ferramenta para orquestrar as implantações de forma declarativa e segura, focando mais na lógica da aplicação do que nos servidores.

### O Pipeline para Funções (FaaS): O Caso do AWS Lambda

No mundo do FaaS (Function-as-a-Service), como o AWS Lambda, usamos frameworks como o AWS Serverless Application Model (AWS SAM) para simplificar nosso trabalho. Pense no AWS SAM como uma extensão do CloudFormation, com uma sintaxe mais curta e amigável para recursos serverless. Em vez de escrever códigos complexos para criar uma função Lambda, você usa um template SAM que já traz as melhores práticas de Infraestrutura como Código.

O pipeline, então, utiliza o comando `sam build` para compilar nosso código e o `sam deploy` para sincronizar o estado da nossa infraestrutura com o que está no template. Para garantir a segurança, a melhor prática é usar o OpenID Connect (OIDC) para que o pipeline obtenha credenciais temporárias para o deploy, evitando o risco de armazenar chaves de acesso de longa duração.

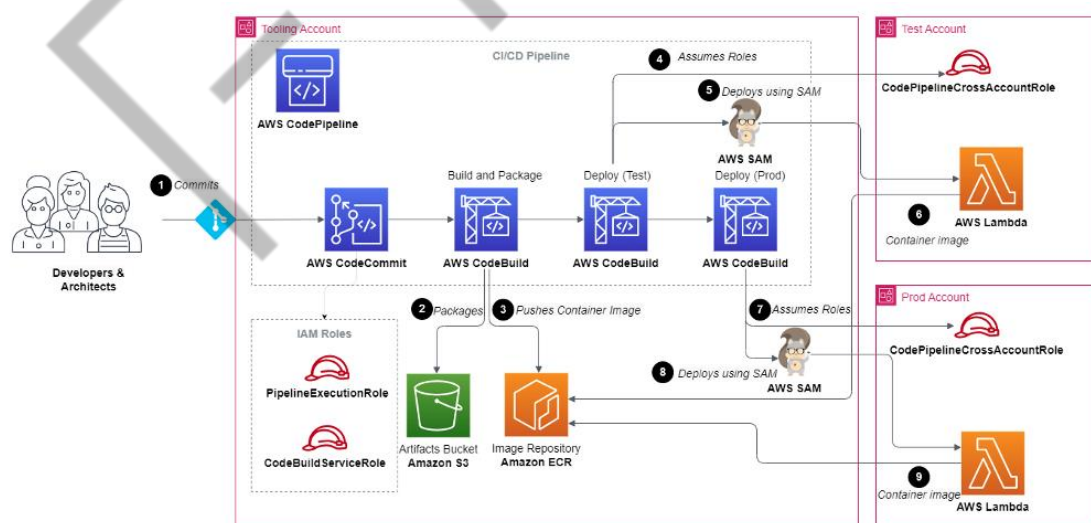


Figura 1 – AWS SAM Lambdas

Fonte:

<https://d2908q01vomqb2.cloudfront.net/1b6453892473a467d07372d45eb05abc2031647a/2023/05/10/Screenshot-2023-05-10-at-12.41.31-PM.png> (2025)

## O Pipeline para Contêineres Serverless

O serverless não se limita apenas a funções. Modelos como os contêineres serverless no AWS ECS Fargate ou no Google Cloud Run oferecem um meio termo perfeito: você tem o controle do ambiente de contêiner, mas sem a necessidade de gerenciar os servidores. Nesses casos, o pipeline de CI/CD se concentra em três etapas principais: a construção da imagem, o envio para um registro de contêiner e a implantação.

- **No AWS Fargate:** o fluxo de trabalho é bastante direto. O pipeline de CI usa ferramentas como o AWS CodeBuild para criar a imagem Docker, a envia para o Amazon ECR e, por fim, o AWS CodePipeline orquestra o deploy no cluster Fargate. Esse fluxo de trabalho automatizado é um exemplo perfeito de como a combinação de serviços gerenciados pode criar um pipeline robusto.

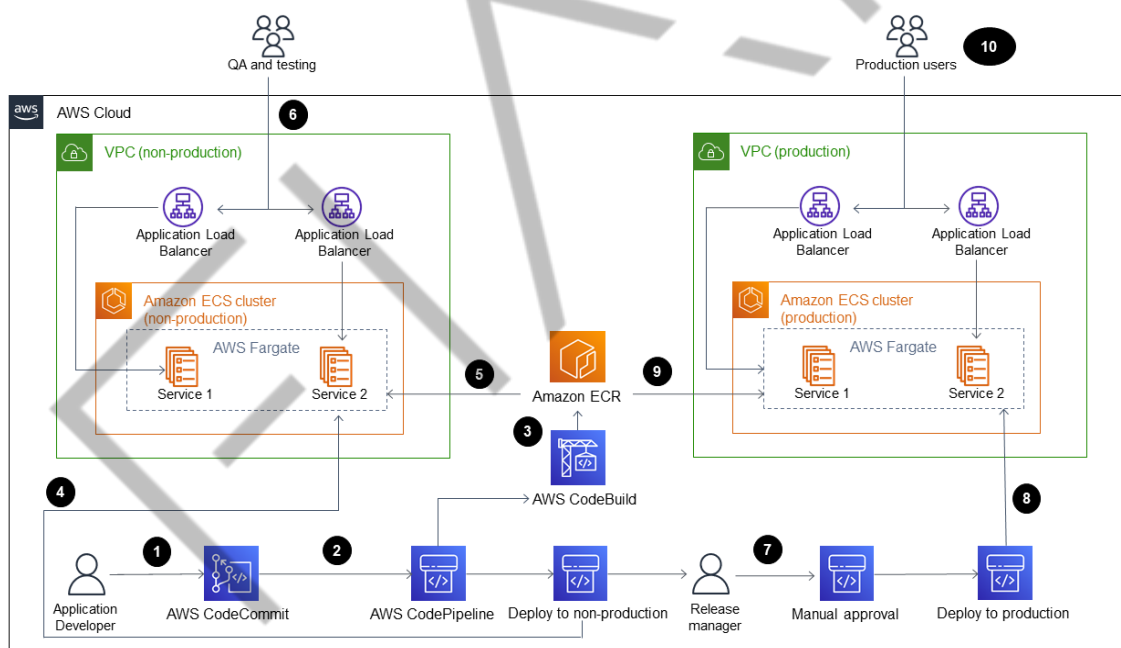


Figura 2 – CI/CD em AWS ECS

Fonte: <https://docs.aws.amazon.com/images/prescriptive-guidance/latest/patterns/images/pattern-img/05ac2cad-408e-433f-8150-0a2b71f63cfd/images/6fa3dbef-88de-4b3f-ae41-dfa90256a058.png>

(2025)

- **No Google Cloud Run:** a abordagem é similar. Você usa o Cloud Build para construir sua imagem, publicá-la no Artifact Registry e, em seguida, o Cloud Run gerencia o deploy final. A simplicidade de configurar um

acionador de build para cada commit e deixar a plataforma cuidar do resto é um grande diferencial para quem busca agilidade.

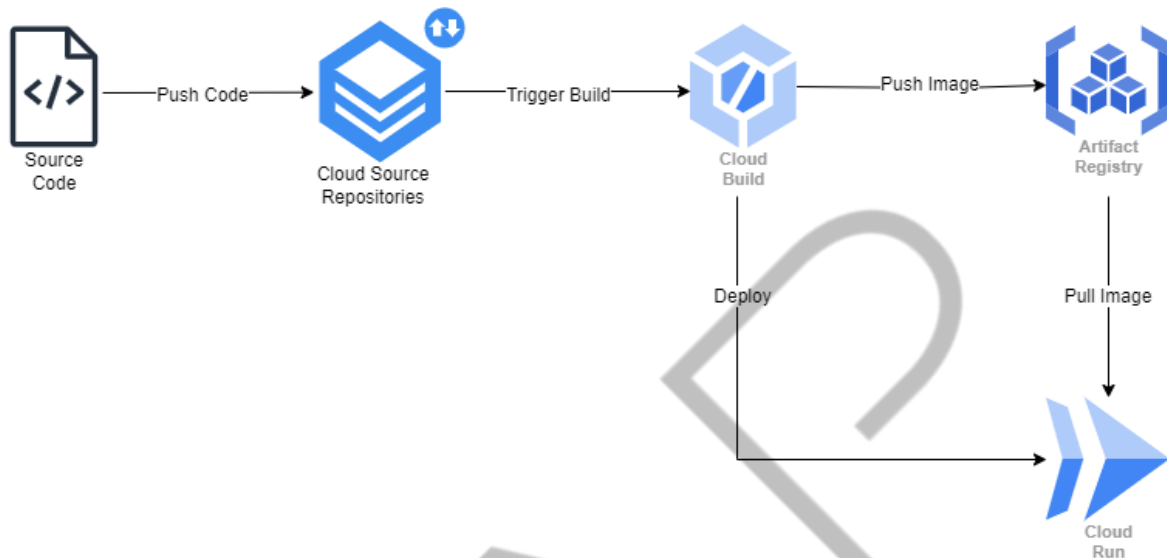


Figura 3 – CI/CD em GCP Cloud Run

Fonte: [https://miro.medium.com/v2/1\\*Rco6R31R\\_Vj\\_LYM2k-lfuw.png](https://miro.medium.com/v2/1*Rco6R31R_Vj_LYM2k-lfuw.png) (2025)

### Estratégias Avançadas e Otimização de Custos

Nós sabemos que a otimização de custos e a performance são cruciais. A boa notícia é que no mundo serverless, eles andam de mãos dadas. No modelo de pagamento por uso, o custo de uma função Lambda, por exemplo, está diretamente ligado ao tempo de execução. Uma das estratégias mais importantes é o "dimensionamento correto" (right-sizing). O sistema da AWS aloca CPU e rede proporcionalmente à memória que você define para a função. Para uma tarefa que exige muito processamento, aumentar a memória pode fazer com que a função termine mais rápido, o que, ironicamente, pode reduzir o custo total.

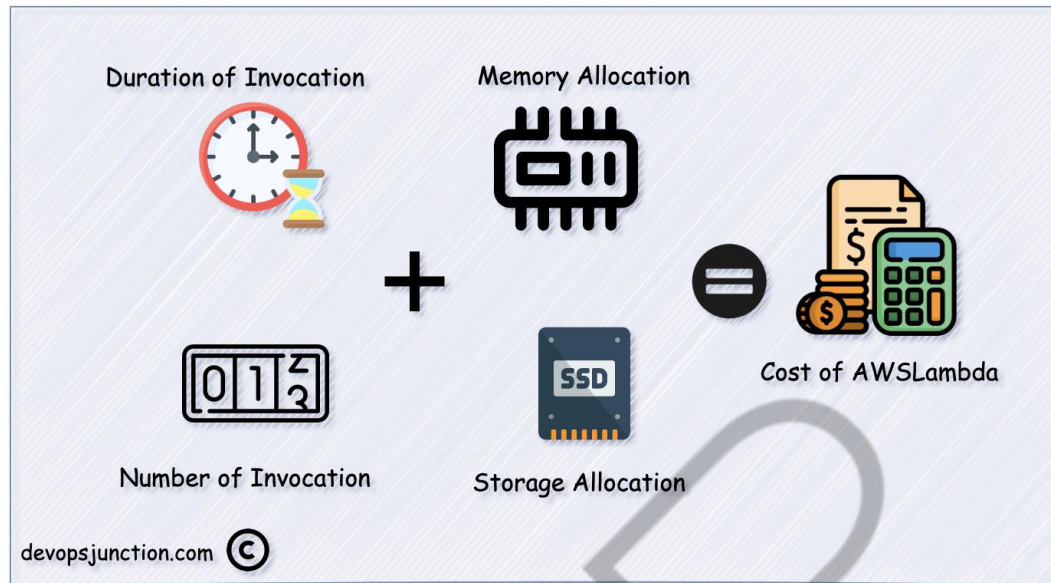


Figura 4 – Custos AWS Lambda

Fonte: <https://www.middlewareinventory.com/wp-content/uploads/2023/02/Screenshot-2023-02-15-at-10.28.32-PM.png> (2025)

Além disso, o próprio pipeline pode ser otimizado, usando caching para evitar instalações repetitivas de dependências ou provisionando ambientes de teste efêmeros para cada pull request, que são destruídos automaticamente após o término dos testes. Essa abordagem evita o desperdício de recursos ociosos, gerando economia significativa em escala.

## MERCADO, CASES E TENDÊNCIAS

A adoção do serverless e a automação de seus pipelines se tornaram um movimento estratégico no mercado. A filosofia cloud-native impulsiona a busca por agilidade e pela entrega de valor mais rápida. Casos de sucesso demonstram como empresas de diversas indústrias superaram desafios de escala e consistência com essa abordagem.

A Amazon, por exemplo, transformou sua infraestrutura de TI interna em um novo modelo de negócio, a AWS. Empresas como Siemens, Genentech, Coinbase e Pismo também utilizaram plataformas serverless da AWS para otimizar operações, aumentar a taxa de sucesso de deploys e acelerar a análise de dados.

Uma tendência crucial que molda o futuro do CI/CD é a ascensão da Engenharia de Plataformas. Vista como a evolução do DevOps, essa disciplina foca na construção de plataformas internas de autoatendimento (Internal Developer Platforms - IDPs) que padronizam e simplificam o ciclo de vida do software.

Ferramentas como o sam pipeline init são estratégicas, pois ajudam as equipes de plataforma a criarem pipelines com as melhores práticas. Essa abordagem libera os desenvolvedores e desenvolvedoras da complexidade do toolchain de CI/CD, permitindo que se concentrem na inovação, o que, em última análise, acelera a produtividade em escala.

## O QUE VOCÊ VIU NESTA AULA?

Nesta aula, a compreensão sobre o pipeline de CI/CD para arquiteturas serverless foi expandida. O texto explorou a natureza estratégica do modelo, que troca a gestão de infraestrutura pela agilidade e pelo foco no código.

Foi apresentado o AWS SAM como a ferramenta central para a definição e orquestração de recursos serverless, agindo como uma camada de abstração que simplifica o uso do poderoso motor do AWS CloudFormation.

A análise se aprofundou nas melhores práticas para construir um pipeline de build, teste e deploy, com ênfase na segurança da automação por meio do uso de credenciais temporárias via OpenID Connect.

Por fim, foram discutidas estratégias avançadas para a otimização de custos e performance, demonstrando como o próprio pipeline pode ser uma ferramenta de economia de recursos.

## REFERÊNCIAS

AWS. **AWS SAM Pipelines.** 2025. Disponível em: <https://serverlessland.com/explore/sam-pipelines>. Acesso em: 07 out. 2025.

AWS. **AWS Serverless Application Model (AWS SAM) Documentation.** 2025. Disponível em: <https://docs.aws.amazon.com/serverless-application-model/>. Acesso em: 07 out. 2025.

CUI, Y.; NAIR, A.; SBARSKI, P. **Serverless Architectures on AWS.** 2. ed. Sebastopol: O'Reilly Media, 2019.

GOOGLE CLOUD. **Creating a CI/CD pipeline with Cloud Run.** 2025. Disponível em: <https://cloud.google.com/dotnet/docs/creating-a-cicd-pipeline-azure-pipelines-cloud-run>. Acesso em: 07 out. 2025.

GOOGLE CLOUD. **What is serverless architecture.** 2025. Disponível em: <https://cloud.google.com/discover/what-is-serverless-architecture>. Acesso em: 07 out. 2025.

HASHNODE. **GitOps for Serverless Functions: Streamlining Deployments for Microservices.** 2024. Disponível em: <https://platformengineer.hashnode.dev/gitops-for-serverless-functions-streamlining-deployments-for-microservices> . Acesso em: 07 out. 2025.

SENTRIO. **Serverless DevOps: How Serverless and DevOps are a Perfect Match.** 2024. Disponível em: <https://sentr.io/blog/serverless-devops/> . Acesso em: 07 out. 2025.

SERVERLESS. **Serverless CI/CD Guide.** 2025. Disponível em: <https://www.serverless.com/guide-ci-cd>. Acesso em: 07 out. 2025.

## **PALAVRAS-CHAVE**

Serverless. AWS SAM. CI/CD. Lambda. Pipelines. DevOps. Otimização de Custos. CloudFormation. Automação.

EMSE



POS TECH